CLAUDE CODE SOURCE STUDY · VOL. 01

소스 진실과 시스템 기준선

복원 저장소의 현재 상태, 검증 가능 범위, 제품 구조를 읽기 위한 공통 좌표계를 먼저 세운다

> 프로젝트의 구조를 외우는 대신, 코드가 어떤 상태와 불변조건을 지키며 결정적인 동작을 보장하는지 이해하기 위한 스터디 보고서다.

이 권에서 얻게 될 통찰

이 권을 마치면 “코드에 존재한다”, “외부 build에서 살아 있다”, “실제로 운영된다”를 서로 다른 주장으로 구분할 수 있다.

Fixed source
Claude Code v2.1.88 복원 snapshot
2ca5ddabfed5f220812ea11f029eda03b21bc4c1
2026-09-20 · Korean study edition

READING BASELINE

OBSERVED

먼저, 지금 보고 있는 시스템의 상태를 고정한다

v2.1.88 npm bundle에서 풀어낸 제3자 복원 snapshot이다. 로컬 clone은 shallow이며 현재 확인 가능한 commit은 하나뿐이다.

구현에서 직접 확인되는 범위

1,884 TS/TSX 파일과 약 51만 줄이 존재하지만 build-time intrinsic, 누락 모듈, 복원용 stub 때문에 공식 내부 소스와 동일하다고 볼 수 없다.

운영을 말할 때 남겨야 할 경계

코드에는 runtime 운영 장치가 보이지만 공식 CI·test·release·feature activation 상태는 이 저장소에서 확인할 수 없다.

이 보고서가 추측으로 채우지 않는 것

공식 모노레포 경계, production feature 값, 누락된 108개 모듈의 구현, 그리고 공식 테스트와 배포 이력은 현재 snapshot만으로 확정할 수 없다. 이후의 설명은 이 공백을 유지한 채, 실제 코드가 보여 주는 구조와 보장만을 다룬다.

읽는 동안 반복할 질문

이 문장은 코드로 관찰한 사실인가, 구조에서 끌어낸 해석인가, 아니면 개선을 위한 제안인가?

SYSTEM READING MAP

OBSERVED

파일이 아니라 책임과 보장의 연결로 읽는다

이 지도는 번호를 따라 구성요소를 외우게 하지 않는다. 각 영역이 어떤 의미를 소유하고, 그 결과가 다음 영역의 판단 근거가 되며, 마지막에 어떤 외부 보장으로 닫히는지를 문장으로 연결한다.

진입 경계	Artifact	`npm bundle → unbundled TypeScript → 연구용 clone`	출처와 변환 손실
			“출처와 변환 손실”가 확인되어야 다음의 “Runtime” 영역이 “현재 호출 관계”를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.
진입 경계	Runtime	`entrypoint → surface orchestrator → query core`	현재 호출 관계
			“현재 호출 관계”가 확인되어야 다음의 “Effects” 영역이 “실행과 authority”를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.
판단 경계	Effects	`Tool → validation/permission → external effect`	실행과 authority
			“실행과 authority”가 확인되어야 다음의 “Durability” 영역이 “기억과 복구”를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.
기록·복구	Durability	`messages → JSONL/DAG → resume projection`	기억과 복구
			“기억과 복구”가 확인되어야 다음의 “Variability” 영역이 “build와 runtime 변형”를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.
의미 연결	Variability	`feature/MACRO/config/GrowthBook`	build와 runtime 변형
			“build와 runtime 변형”가 확인되어야 다음의 “Operations” 영역이 “관찰 가능한 운영 코드”를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.
기록·복구	Operations	`analytics/migration/managed settings`	관찰 가능한 운영 코드
			이 책임은 앞선 판단을 외부에서 관찰 가능한 결과로 닫는다. 이 지점의 실패 처리가 모호하면 앞의 모든 단계가 성공했더라도 시스템 전체는 정산되지 않은 상태로 남는다.

선을 따라갈 때에는 구현 이름보다 인접한 두 책임 사이에서 전달되는 identity, state, completion을 확인한다. 그 셋 중 하나가 사라지는 지점이 구조적 결함이 나타날 가능성이 가장 높은 경계다.

01 무엇을 분석하고 있는가

소스의 출처와 신뢰 경계를 고정하지 않으면 이후의 모든 아키텍처 설명이 과장된다.

이 장의 논증은 다음 질문을 차례로 해결한다

해석	**책임 연결** 공식 개발 저장소가 아니라 복원 산출물이다	**구분**	**책임 연결** 네 가지 주장 수준
코드	**책임 연결** 복원용 TypeScript 설정이 품질 인증은 아니다		

코드 독해의 시작점

`README.md` → `package.json` → `tsconfig.json` → `.git`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

CONCEPT IN CONTEXT

OBSERVED

공식 개발 저장소가 아니라 복원 산출물이다

이 저장소는 배포된 단일 bundle을 TypeScript 모듈로 풀어낸 연구용 projection이다.

코드에서 읽어낼 수 있는 단서

출발 단서	README는 npm package v2.1.88에서 추출했다고 명시한다.
연결되는 단서	Git history는 shallow clone의 merge commit 하나만 제공한다.
연결되는 단서	package metadata와 source import graph는 production dependency manifest로 보기 어렵다.

이 단서들을 하나의 설계로 묶으면

파일 구조가 풍부하다는 사실과 개발 원본이라는 주장은 동일하지 않다. 주석과 이름은 중요한 evidence지만 공식 설계 의도나 운영 활성화를 자동으로 증명하지 않는다.

시스템 이해에 남는 결론

보고서의 모든 주장에는 observed·repository claim·inference·unknown 중 하나의 상태가 붙어야 한다.

이 설명을 확인할 코드 README.md:1-34 package.json:1-20

DISTINCTION TABLE

OBSERVED

네 가지 주장 수준

같은 문장 안에서 관찰과 추론을 섞지 않기 위한 판정표다.

표지	말할 수 있는 것	말하면 안 되는 것
OBSERVED	파일·타입·호출·분기가 snapshot에 존재	production에서 활성이라고 단정
REPOSITORY CLAIM	README/docs가 그렇게 주장	공식 Anthropic 사실로 승격
INFERENCE	여러 관찰을 묶은 제한된 해석	유일한 설계 의도로 단정
UNKNOWN	현재 자료로 검증 불가	빈칸을 그럴듯한 구현으로 채움

이 표를 읽는 관점

이 표지는 문서 장식이 아니라 반증 가능 범위를 지정한다.

이 설명을 확인할 코드 README.md:1-34

SOURCE READING · 1/2

OBSERVED

복원용 TypeScript 설정이 품질 인증은 아니다

컴파일 설정은 읽기와 best-effort 재구성을 돕지만 원본 개발 환경을 재현하지 않는다.

복원 저장소의 compiler options tsconfig.json:1-35

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": false,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "declaration": true,
    "declarationMap": true,
    "sourceMap": true,
    "jsx": "react-jsx",
    "outDir": "dist",
    "rootDir": "src",
    "baseUrl": ".",
    "paths": {
      "bun:bundle": ["stubs/bun-bundle.ts"],
      "src/*": ["src/*"]
    },
    "types": ["node"],
    "lib": ["ES2022", "DOM"],
    "allowImportingTsExtensions": false,
… 5 lines omitted; use the source link for the complete range …
  ],
  "exclude": [
    "node_modules",
    "dist"
  ]
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	strict가 false다.
그 표현이 만든 조건	skipLibCheck가 true다.
그 표현이 만든 조건	bun:bundle을 stub path로 연결한다.
그 표현이 만든 조건	src와 stubs를 함께 include한다.

언어와 런타임은 여기까지 보장한다

TypeScript의 타입 검사는 설정에 따라 보증 범위가 달라진다. strict off는 타입이 없다는 뜻은 아니지만 null·variance·implicit any 등에 대한 보증이 약해진다.

프로젝트가 이 메커니즘에 맡긴 책임

이 설정은 "원본과 같은 빌드"보다 "복원 코드를 도구로 탐색"하는 목적에 가깝다.

경계를 잘못 읽을 때 생기는 위험

typecheck 성공을 production equivalence로 해석하면 안 된다.

이 설명을 확인할 코드 tsconfig.json:1-35

02 복원 build는 어떤 알고리즘을 쓰는가

build 성공과 의미 복원은 다르다. 변환·제거·stub 생성의 순서를 알고 결과를 해석해야 한다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	책임 연결 복사본 변환과 반복 stub 생성	알고리즘	책임 연결 원본을 직접 고치는 prepare-src
코드	의미 연결 feature stub은 모든 기능의 기본값이 아니다	구분	책임 연결 기능 상태 판정표

코드 독해의 시작점

`scripts/prepare-src.mjs` → `scripts/build.mjs` → `stubs/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A01 · 독자의 질문

복사본 변환과 반복 stub 생성은 어떤 시스템 문제를 닫는가

복원된 source를 읽으려는데 원래 빌드 환경의 intrinsic과 비공개 모듈 때문에 도구가 파일을 해석하지 못한다. 이때 build 성공은 제품이 같은 의미로 실행된다는 뜻이 아니라, 분석 가능한 변형을 얻었다는 뜻부터 분리해야 한다.

코드가 필요해지는 이유

Bun 전용 intrinsic과 누락 모듈이 있는 source를 Node/esbuild가 읽을 수 있는 형태로 만든다.

OBSERVED — scripts/build.mjs가 원본과 분리된 build-src working copy, 치환 횟수, 각 round의 missing module 집합을 소유한다.

INFERENCE — 이 알고리즘은 제품 runtime이 아니라 source 복원 경계에 놓인다. 따라서 산출물은 구현 탐색의 입력이지 production parity의 증거가 아니다.

이 절에서 답할 결정 질문

복사·치환·stub 반복은 무엇을 보존하고 무엇을 의도적으로 포기하며, 어느 시점에 반복을 멈추는가?

이 코드를 읽기 전에 잡을 개념

고정점 반복과 부분 평가. 현재 오류에서 새 missing module을 발견해 stub을 보태고 다시 build하는 반복은 더 이상 새 항목이 생기지 않을 때 고정점에 도달한다. compile-time literal 치환은 실행 전 이미 알려진 값을 source에 반영하는 부분 평가다.

52–117행에서는 round마다 missing set이 실제로 줄거나 안정되는지, 변환이 working copy에만 적용되는지 확인한다.

이 설명을 확인할 코드 scripts/build.mjs:52-117 scripts/build.mjs:130-229

ALGORITHM A01 · 실제 코드

OBSERVED

복사본 변환과 반복 stub 생성의 결정점은 어디에 있는가

복원 builder는 원본을 실행 가능하게 증명하는 compiler가 아니라 실패를 줄여 가는 best-effort reconstruction loop다.

입력 표현. src tree, 고정 VERSION, 그리고 esbuild 오류의 missing module 목록

보존되는 작업 상태. build-src working copy, transformCount, 그리고 round별 missing set

copy와 compile-time 치환

scripts/build.mjs:52-117

```
52    // ═══════════════════════════════════════════════════════════════
53    // PHASE 1: Copy source
54    // ═══════════════════════════════════════════════════════════════
55
56    await rm(BUILD, { recursive: true, force: true })
57    await mkdir(BUILD, { recursive: true })
58    await cp(join(ROOT, 'src'), join(BUILD, 'src'), { recursive: true })
59    console.log('✅ Phase 1: Copied src/ → build-src/')
60
61    // ═══════════════════════════════════════════════════════════════
62    // PHASE 2: Transform source
63    // ═══════════════════════════════════════════════════════════════
64
65    let transformCount = 0
… 49 lines omitted; use the source link for the complete range …
115     }
116   }
117   console.log(`✅ Phase 2: Transformed ${transformCount} files`)
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A01 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

src tree, 고정 VERSION, 그리고 esbuild 오류의 missing module 목록은 처음부터 신뢰된 실행 상태가 아니다. build-src working copy, transformCount, 그리고 round별 missing set가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** src를 build-src로 복사한다. 이 결과가 다음 판단 "feature('X')를 false로 치환한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** feature('X')를 false로 치환한다. 이 결과가 다음 판단 "MACRO 값을 literal로 바꾼다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** MACRO 값을 literal로 바꾼다. 이 결과가 다음 판단 "bun:bundle import를 제거한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** bun:bundle import를 제거한다. 이 결과가 다음 판단 "bundle 실패에서 누락 import를 수집하고 stub을 만든 뒤 재시도한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** bundle 실패에서 누락 import를 수집하고 stub을 만든 뒤 재시도한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

원본 src는 build.mjs 경로에서 수정하지 않는다. feature-gated branch는 외부 복원 build에서 제거된다. stub은 symbol resolution만 제공하고 의미를 제공하지 않는다.

ALGORITHM A01 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **구성·변형·성능의 질문으로 읽기.** 원본 src는 build.mjs 경로에서 수정하지 않는다.
- **내구성·복구의 질문으로 읽기.** feature-gated branch는 외부 복원 build에서 제거된다.
- **구성·변형·성능의 질문으로 읽기.** stub은 symbol resolution만 제공하고 의미를 제공하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 정규식이 예상하지 못한 feature 표현
- stub export shape 불일치
- build 성공 뒤 runtime 의미 부재

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 정확한 Bun build 환경을 재현하지 못하는 대신, 읽기 가능한 최대 범위의 외부 variant를 만든다.

RECOMMENDATION 변환 manifest와 stub provenance를 남기고 “syntactic build”와 “semantic smoke”를 별도 상태로 기록한다.

ALGORITHM A01 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

빌드 반복은 실패 로그를 다음 입력으로 사용하며, 성공은 esbuild가 graph를 닫았다는 관찰이다.

실패 뒤 남는 상태

새 missing set이 계속 바뀌거나 stub shape가 import 기대와 다르면 syntactic graph는 닫히지 않는다. build가 끝나도 stub 호출의 runtime 의미는 남지 않는다.

OBSERVED	고정 snapshot에는 working copy 복사, compile-time 치환, 오류 기반 stub 반복이 존재한다.
CONDITIONAL	해당 transform pattern에 맞는 feature 표현에서는 외부 도구가 더 많은 source를 읽을 수 있다.
UNKNOWN	원래 Bun 빌드와 생성 bundle의 행동 동등성은 이 source만으로 확인할 수 없다.
NEXT READ	변경자는 transform manifest와 실제 smoke 경로를 대조하고, 새 missing module이 등장하면 stub provenance부터 재검증해야 한다.

다시 적용할 질문

1. 새 intrinsic 하나를 치환했을 때 build-src와 원본 중 어느 쪽의 digest만 달라져야 하는가?
2. stub을 실제 모듈로 바꾸면 fixed-point 종료 조건과 semantic smoke의 의미가 어떻게 달라지는가?

ALGORITHM A02 · 독자의 질문

원본을 직접 고치는 prepare-src은 어떤 시스템 문제를 닫는가

편한 검사 명령 하나를 위해 source 자체를 자동 치환할 수 있지만, 분석 대상이 조용히 달라지면 이후 모든 줄 근거가 불안정해진다.

코드가 필요해지는 이유

Bun import와 MACRO를 일반 TypeScript 도구가 처리할 수 있게 바꾼다.

OBSERVED — scripts/prepare-src.mjs가 파일별 changed flag와 실행 시 계산한 BUILD_TIME을 가진 채 src 파일을 직접 쓴다.

INFERENCE — 이 경로는 읽기 전용 분석 workflow와 양립하지 않는다. 복원 도구로는 존재하지만 고정 source 증거를 만드는 경로에서는 실행하지 않아야 한다.

이 절에서 답할 결정 질문

in-place 변환의 편의와 source provenance 사이의 손실을 어디에서 관찰할 수 있는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **source-to-source 변환과 멱등성.** 같은 입력에 변환을 두 번 적용해도 결과가 같아야 안전하게 반복할 수 있다. 그러나 매 실행 시각을 넣는 변환은 내용이 계속 달라져 완전한 멱등성을 깨뜨린다.
>
> 36–76행의 patchFile이 string·comment 경계를 아는 parser인지, regex인지와 BUILD_TIME 생성 시점을 함께 본다.

이 설명을 확인할 코드 scripts/prepare-src.mjs:36-76

ALGORITHM A02 · 실제 코드

OBSERVED

원본을 직접 고치는 prepare-src의 결정점은 어디에 있는가

별도 build 경로와 달리 prepare-src는 source tree 자체를 rewrite한다.

입력 표현. src의 모든 ts/tsx, 그리고 macro replacement table

보존되는 작업 상태. 파일별 changed flag, 그리고 실행 시점 BUILD_TIME

in-place patchFile

scripts/prepare-src.mjs:36-76

```
36    function patchFile(filePath) {
37      let src = fs.readFileSync(filePath, 'utf8')
38      let changed = false
39
40      // 1. Replace `import { feature } from 'bun:bundle'` / `"bun:bundle"`
41      if (src.includes("from 'bun:bundle'") || src.includes('from "bun:bundle"')) {
42        src = src.replace(/import\s*\{\s*feature\s*\}\s*from\s*['"]bun:bundle['"]/g,
43          "import { feature } from '../stubs/bun-bundle.js'")
44        // Fix relative depth based on file location
45        const rel = path.relative(SRC, path.dirname(filePath))
46        const depth = rel ? '../'.repeat(rel.split('/').length) : ''
47        if (depth) {
48          src = src.replace("from '../stubs/bun-bundle.js'", `from '${depth}stubs/bun-bundle.js'`)
49        }
… 24 lines omitted; use the source link for the complete range …
74        return true
75      }
76      return false
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A02 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

src의 모든 ts/tsx, 그리고 macro replacement table은 처음부터 신뢰된 실행 상태가 아니다. 파일별 changed flag, 그리고 실행 시점 BUILD_TIME가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 파일을 읽는다. 이 결과가 다음 판단 "bun:bundle import를 상대 stub import로 바꾼다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** bun:bundle import를 상대 stub import로 바꾼다. 이 결과가 다음 판단 "MACRO expression을 문자열로 바꾼다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** MACRO expression을 문자열로 바꾼다. 이 결과가 다음 판단 "changed인 파일을 원위치에 쓴다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** changed인 파일을 원위치에 쓴다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

재실행은 대부분 동일 결과를 내야 한다. 분석 provenance를 보존하려면 실행 전 commit 상태가 필요하다.

ALGORITHM A02 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **구성·변형·성능의 질문으로 읽기.** 재실행은 대부분 동일 결과를 내야 한다.
- **내구성·복구의 질문으로 읽기.** 분석 provenance를 보존하려면 실행 전 commit 상태가 필요하다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- BUILD_TIME 때문에 완전한 idempotence가 깨질 수 있음
- regex rewrite가 string/comment 경계를 오판
- 분석 원본이 조용히 변형

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 간단한 `npm run check` 경험을 위해 변환 결과를 source에 직접 반영한다.

RECOMMENDATION read-only 분석 clone에서는 실행하지 않고, 항상 temporary working copy를 대상으로 삼는다.

ALGORITHM A02 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

모든 TS/TSX를 순회해 pattern을 치환하고 changed 파일만 기록한다.

실패 뒤 남는 상태

regex 오판이나 중간 실패는 일부 파일만 변한 working tree를 남길 수 있다.

OBSERVED	in-place patchFile과 실행 시점 BUILD_TIME 치환이 구현돼 있다.
CONDITIONAL	예상한 macro와 import 문법에 대해서만 일반 TypeScript 도구가 읽을 가능성이 높아진다.
UNKNOWN	모든 문자열·주석·새 문법에서 rewrite가 안전한지는 확인되지 않는다.
NEXT READ	분석 환경에서는 실행하지 말고 temporary copy에서 before/after diff와 두 번째 실행 결과를 비교해야 한다.

다시 적용할 질문

1. 변환을 두 번 실행했을 때 BUILD_TIME 외에 달라지는 파일이 있다면 어떤 전제가 깨졌는가?
2. regex를 AST transform으로 바꾸면 provenance와 실패 원자성을 어떻게 기록할 것인가?

SOURCE READING · 1/2

OBSERVED

feature stub은 모든 기능의 기본값이 아니다

복원 stub의 false는 외부 분석 build의 선택이지 내부 production 값에 대한 evidence가 아니다.

복원용 feature() stubs/bun-bundle.ts:1-4

```
1  // Stub for bun:bundle — feature() is compile-time in Bun; replaced by build script
2  export function feature(_flag: string): boolean {
3    return false
4  }
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	모든 flag 이름을 무시한다.
그 표현이 만든 조건	항상 false를 반환한다.
그 표현이 만든 조건	build.mjs는 호출 자체를 false literal로 바꾼다.

언어와 런타임은 여기까지 보장한다

compile-time intrinsic은 bundler가 분기를 제거할 수 있게 한다. runtime flag와 달리 제거된 모듈은 배포 뒤 켤 수 없다.

프로젝트가 이 메커니즘에 맡긴 책임

이 저장소에서 누락 import가 보이는 이유와 실제 npm bundle에서 branch가 사라지는 이유를 함께 설명한다.

경계를 잘못 읽을 때 생기는 위험

feature() 호출 수를 활성 기능 수로 집계해서는 안 된다.

이 설명을 확인할 코드 stubs/bun-bundle.ts:1-4

DISTINCTION TABLE

OBSERVED

기능 상태 판정표

코드 존재와 운영 활성화를 분리한다.

상태	확인 방법	보고서 표현
included	구현 파일과 호출 경로 존재	snapshot에 구현 존재
compile-time excluded	feature gate + 외부 false	외부 variant에서 제거 가능
runtime gated	GrowthBook/config 평가 지점	실행 중 값에 따라 달라짐
referenced but missing	import는 있으나 파일 없음	구현 검증 불가
stubbed	복원 script가 placeholder 생성	실행 의미 없음
operationally active	배포 관찰 필요	이 저장소만으로 unknown

이 표를 읽는 관점

각 기능은 하나 이상의 축을 가질 수 있다. included이면서 runtime gated일 수 있다.

이 설명을 확인할 코드 scripts/build.mjs:80-117

03 공통 시스템 기준선

이후 다섯 권은 동일한 실제 호출 관계와 상태 지도를 공유한다.

이 장의 논증은 다음 질문을 차례로 해결한다

사례	진입 경계 공통 runtime 지도	구분	책임 연결 현재 구현·설계 의도·개선안을 분리해 읽기
해석	책임 연결 큰 파일은 결함 점수가 아니라 변경 압력 지도다	구분	책임 연결 이후 권별 질문

코드 독해의 시작점

`src/entrypoints` → `src/main.tsx` → `src/screens/REPL.tsx` → `src/QueryEngine.ts` → `src/query.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SELECTIVE EXECUTION TRACE

OBSERVED

공통 runtime 지도

표면은 여러 개지만 turn core는 query()에 수렴한다.

진입 경계

CLI routing

특수 process와 일반 실행을 먼저 고른다.

이때 남은 상태와 판단이 “Surface orchestration”에서 사용할 전제가 된다.

책임 연결

Surface orchestration

REPL 또는 print/headless가 session 입력과 상태를 준비한다.

이때 남은 상태와 판단이 “Turn core”에서 사용할 전제가 된다.

책임 연결

Turn core

query()가 model과 tool phase를 반복한다.

이때 남은 상태와 판단이 “Effect runtime”에서 사용할 전제가 된다.

진입 경계

Effect runtime

Tool pipeline이 validation·permission·call을 수행한다.

이때 남은 상태와 판단이 “Durability/projection”에서 사용할 전제가 된다.

기록·복구

Durability/projection

transcript, SDK event, UI state가 서로 다른 표현을 만든다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

공유 core가 있다는 사실과 모든 표면이 같은 state owner를 쓴다는 주장을 구분한다.

어디에서 설명이 틀어지는가

REPL은 QueryEngine을 우회하므로 두 orchestration 경로가 drift 할 수 있다.

이 설명을 확인할 코드 src/screens/REPL.tsx:2788-2804 src/QueryEngine.ts:675-704

DISTINCTION TABLE

OBSERVED

현재 구현·설계 의도·개선안을 분리해 읽기

보고서의 모든 구조 그림은 어느 열에 속하는지 표시한다.

구분	현재 snapshot	개선 방향
Conversation shell	REPL 직접 query + headless QueryEngine	공통 application runtime
Tool contract	effect·policy·presentation 결합	port 분리와 compatibility facade
Transcript	parent chain + recovery pass	명시적 replay invariant
Feature	compile/runtime gate 혼합	variant catalog와 activation matrix

이 표를 읽는 관점

오른쪽 열은 현재 코드의 설명이 아니라 별도의 recommendation이다.

CONCEPT IN CONTEXT

INFERENCE

큰 파일은 결함 점수가 아니라 변경 압력 지도다

main, REPL, messages, sessionStorage 같은 거대 파일은 서로 다른 변경 이유가 만나는 장소를 보여 준다.

코드에서 읽어낼 수 있는 단서

출발 단서	REPL.tsx는 UI와 turn orchestration을 함께 가진다.
연결되는 단서	messages.ts는 canonical message, API projection, UI helper를 함께 가진다.
연결되는 단서	sessionStorage.ts는 journal write, load, DAG repair, metadata를 함께 가진다.

이 단서들을 하나의 설계로 묶으면

라인 수만으로 분해하면 hidden state와 ordering constraint가 더 깊이 숨을 수 있다.

시스템 이해에 남는 결론

리팩터링 단위는 함수 크기가 아니라 state ownership과 보존 불변조건으로 정해야 한다.

이 설명을 확인할 코드 src/screens/REPL.tsx:572-640 src/utils/sessionStorage.ts:2063-2117

DISTINCTION TABLE

OBSERVED

이후 권별 질문

여섯 권이 서로 다른 분류가 아니라 하나의 시스템을 확대해 보는 순서다.

권	중심 질문	끝까지 보존할 것
02	누가 session과 turn 상태를 소유하는가	surface별 차이
03	한 turn은 어떤 protocol로 닫히는가	event와 terminal
04	제안은 어떻게 권한 있는 effect가 되는가	최종 input identity
05	부분 실행 뒤 어떻게 재개하는가	journal과 workspace truth
06	제품 변형과 확장을 어떻게 운영하는가	provenance와 compatibility

이 표를 읽는 관점

모든 권은 “파일 목록”이 아니라 변경 시 보존할 불변조건을 제공한다.

04 전체 범위는 파일 수가 아니라 제품 책임으로 닫는다

1,884개 파일을 모두 한 번씩 언급하는 것은 coverage가 아니다. 실행을 결정하는 중심 경로와 그 경로를 둘러싼 제품 shell·운영 장치를 책임 단위로 연결해야 독자가 현재 시스템과 아직 모르는 영역을 동시에 복원할 수 있다.

이 장의 논증은 다음 질문을 차례로 해결한다

해석	책임 연결 요청한 분석 대상은 stupidhumans 저장소의 main snapshot이다	구분	책임 연결 1,884개 모듈의 분포를 책임 지도로 읽는다
구분	책임 연결 프로젝트 전체를 세 층으로 읽는 coverage 지도	사례	책임 연결 전체 범위 감사는 이렇게 읽는다

코드 독해의 시작점

`README.md` → `src/**` → `docs/ko/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

CONCEPT IN CONTEXT

OBSERVED

요청한 분석 대상은 stupidhumans 저장소의 main snapshot이다

이 보고서가 설명하는 대상은 지정된 GitHub 저장소를 commit 2ca5ddab에 고정한 로컬 clone이며, Anthropic의 공식 개발 모노레포나 이후 버전 일반론이 아니다.

코드에서 읽어낼 수 있는 단서

출발 단서	로컬 clone의 origin은 https://github.com/stupidhumans/claude-code-source-code.git 이다.
연결되는 단서	README는 @anthropic-ai/claude-code 2.1.88의 단일 bundle에서 TypeScript source를 추출했다고 스스로 규정한다.
연결되는 단서	README는 약 1,884개 TS/TSX 파일과 누락된 108개 feature-gated module을 함께 기록한다.
연결되는 단서	따라서 관찰 가능한 파일·타입·분기는 OBSERVED이지만, 누락 module의 구현과 production activation은 UNKNOWN이다.

이 단서들을 하나의 설계로 묶으면

이 범위 설정은 보고서를 작게 만드는 제한이 아니라 주장의 정확한 모집단을 정한다. 저장소가 제공하는 CLI 구현 전체를 읽되, 저장소 바깥의 공식 의도·backend retention·internal build 값을 코드 빈칸에 채우지 않는다.

시스템 이해에 남는 결론

새로운 source revision이나 다른 fork를 섞으려면 별도 source profile과 digest-bound build가 필요하다.

이 설명을 확인할 코드 README.md:1-15 README.md:59-172 README.md:189-220

DISTINCTION TABLE

OBSERVED

1,884개 모듈의 분포를 책임 지도로 읽는다

디렉터리 개수는 중요도 순위가 아니라 어디에서 변경 압력과 플랫폼 세부가 퍼져 있는지 묻기 위한 탐색 지도다.

관찰된 분포	현재 보고서에서 묶는 책임	독자가 확인할 질문
utils 564	인증·설정·권한·네트워크·메시지·내구성의 공용 메커니즘	이름이 utils여도 실제 policy owner인가, 단순 adapter인가?
components 389 + ink 96	React interaction과 terminal layout/frame/diff/write	화면 state와 물리 terminal state 사이 commit은 어디인가?
commands 189 + tools 184	사용자 command와 model-proposed effect capability	발견·노출·권한·실행을 누가 각각 결정하는가?
services 130	API·analytics·compact·plugin·MCP·동기화	외부 transport와 canonical state 사이 anti-corruption 경계는 어디인가?
hooks 104	React lifecycle과 application/service event의 접점	hook cleanup이 실제 resource lifetime까지 닫는가?
bridge 31 + remote 4	local 의미의 network projection	재연결 뒤 중복·gap·미결 decision을 어떻게 정산하는가?
tasks 12 + root task modules	background work·team task·nested agent lifecycle	상태 파일, process, UI 중 어느 것이 truth인가?

이 표를 읽는 관점

파일 수가 많은 영역을 더 중요한 것으로 점수화하지 않는다. 각 행은 뒤 권의 source reading과 연결되고, 연결되지 않은 leaf는 scope ledger에서 contextual coverage로 남긴다.

이 설명을 확인할 코드 README.md:223-320

DISTINCTION TABLE

OBSERVED

프로젝트 전체를 세 층으로 읽는 coverage 지도

agent loop만으로 제품 전체를 대표하지 않도록 core, product shell, operations의 서로 다른 실패 질문을 유지한다.

층	포함하는 책임	이 보고서의 배치
결정적 runtime core	입력→context→model stream→tool proposal→permission/effect→settlement	2-5권에서 코드와 알고리즘으로 상세 추적
사용자·플랫폼 shell	terminal renderer, command, instruction, model policy, auth, IDE/browser/voice, team/worktree	2-6권의 별도 학습 단위로 owner와 activation을 설명
운영·진화 장치	telemetry, privacy boundary, update, migration, remote policy, plugin/MCP	6권에서 control/data plane과 UNKNOWN을 분리
복원·연구 경계	bundle extraction, stub, missing feature module, repository-authored reports	1권에서 source claim과 직접 관찰을 구분

이 표를 읽는 관점

"핵심 loop를 상세히 설명했다"와 "제품을 구성하는 책임을 빠짐없이 지도화했다"는 다른 완료 조건이다. 상세도는 risk와 결정성에 따라 달라도 모든 책임은 최소한 owner·activation·effect·unknown을 가져야 한다.

SELECTIVE EXECUTION TRACE

OBSERVED

전체 범위 감사는 이렇게 읽는다

독자는 scope ledger의 각 domain을 단순 체크 표시가 아니라 다음 질문의 연쇄로 검증한다.

존재
어떤 파일·타입·분기가 이 책임의 존재를 직접 보여 주는가?
이때 남은 상태와 판단이 "소유"에서 사용할 전제가 된다.

책임 연결

소유
그 상태와 최종 결정을 어느 module·process·durable store가 소유하는가?
이때 남은 상태와 판단이 "활성"에서 사용할 전제가 된다.

책임 연결

활성
symbol 존재, bundle 포함, runtime configured, 실제 exercised를 구분했는가?
이때 남은 상태와 판단이 "전이"에서 사용할 전제가 된다.

책임 연결

전이
입력이 어떤 representation과 state transition을 거쳐 effect 또는 projection이 되는가?
이때 남은 상태와 판단이 "실패"에서 사용할 전제가 된다.

책임 연결

실패
부분 성공·race·stale state·정보 손실 때 무엇이 남고 무엇을 복구하는가?
이때 남은 상태와 판단이 "경계"에서 사용할 전제가 된다.

책임 연결

경계
source가 답하지 못하는 backend·배포·누락 module 질문을 UNKNOWN으로 멈췄는가?
이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

모든 domain이 같은 분량이나 page template를 가져야 한다는 뜻은 아니다. 결정적 알고리즘은 A–K 연속 서술로, 구조적 책임은 code·matrix·walkthrough의 결합으로 설명한다.

어디에서 설명이 틀어지는가

title 존재만으로 coverage를 닫거나 README의 주장만으로 운영 사실을 확정하면 감사는 다시 실패한다.

이 설명을 확인할 코드 README.md:17-63 README.md:202-220

THEORY ATLAS · 1/1

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

내구성·복구

2개 알고리즘 · 2개 표현

부분 기록, 재시작, 버전 변화가 있어도 이전 사실을 잃거나 같은 effect를 다시 확정하지 않도록 만드는 복구 계약을 설명한다.

이 권에서 나타나는 방식. A01 복사본 변환과 반복 stub 생성과 A02 원본을 직접 고치는 prepare-src에서 이 관점이 결정적인 이유는 "원본 src는 build.mjs 경로에서 수정하지 않는다. 또한 재실행은 대부분 동일 결과를 내야 한다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. fixed-point retry(A01), idempotence(A02)

스터디 질문. 중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

권한·신뢰 경계

1개 알고리즘 · 1개 표현

실행 가능성, 허용 여부, 데이터 노출을 분리하고 신뢰하지 않은 입력이 effect 권한으로 곧바로 승격되지 않게 한다.

이 권에서 나타나는 방식. A02 원본을 직접 고치는 prepare-src에서 이 관점이 결정적인 이유는 "재실행은 대부분 동일 결과를 내야 한다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. provenance(A02)

스터디 질문. 누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

구성·변형·성능

2개 알고리즘 · 3개 표현

동일한 핵심 의미를 여러 실행 표면과 제품 변형에 조립하면서 비용과 의존성의 위치를 통제하는 방법을 다룬다.

이 권에서 나타나는 방식. A01 복사본 변환과 반복 stub 생성과 A02 원본을 직접 고치는 prepare-src에서 이 관점이 결정적인 이유는 "원본 src는 build.mjs 경로에서 수정하지 않는다. 또한 재실행은 대부분 동일 결과를 내야 한다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. partial evaluation(A01), dead-code elimination(A01), source-to-source transformation(A02)

스터디 질문. 조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

ALGORITHM INDEX

OBSERVED

이 권의 결정적 알고리즘과 보장

알고리즘 이름보다 마지막 열을 먼저 읽는다. 프로젝트에서 중요한 것은 계산 기법 자체보다 그 계산으로 어떤 시스템 조건을 계속 참으로 만드는가이기 때문이다.

ID	문제와 구현	대표 불변조건
A01	**복사본 변환과 반복 stub 생성** Bun 전용 intrinsic과 누락 모듈이 있는 source를 Node/esbuild가 읽을 수 있는 형태로 만든다.	원본 src는 build.mjs 경로에서 수정하지 않는다.
A02	**원본을 직접 고치는 prepare-src** Bun import와 MACRO를 일반 TypeScript 도구가 처리할 수 있게 바꾼다.	재실행은 대부분 동일 결과를 내야 한다.

SOURCE EVIDENCE INDEX · 1/1

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
README.md:1-34	공식 개발 저장소가 아니라 복원 산출물이다	explain
package.json:1-20	공식 개발 저장소가 아니라 복원 산출물이다	explain
tsconfig.json:1-35	복원용 TypeScript 설정이 품질 인증은 아니다	code
scripts/build.mjs:52-117	복사본 변환과 반복 stub 생성	algorithm
scripts/build.mjs:130-229	복사본 변환과 반복 stub 생성	algorithm
scripts/prepare-src.mjs:36-76	원본을 직접 고치는 prepare-src	algorithm
stubs/bun-bundle.ts:1-4	feature stub은 모든 기능의 기본값이 아니다	code
scripts/build.mjs:80-117	기능 상태 판정표	matrix
src/screens/REPL.tsx:2788-2804	공통 runtime 지도	walkthrough
src/QueryEngine.ts:675-704	공통 runtime 지도	walkthrough
src/screens/REPL.tsx:572-640	큰 파일은 결함 점수가 아니라 변경 압력 지도다	explain
src/utils/sessionStorage.ts:2063-2117	큰 파일은 결함 점수가 아니라 변경 압력 지도다	explain
README.md:1-15	요청한 분석 대상은 stupidhumans 저장소의 main snapshot이다	explain
README.md:59-172	요청한 분석 대상은 stupidhumans 저장소의 main snapshot이다	explain
README.md:189-220	요청한 분석 대상은 stupidhumans 저장소의 main snapshot이다	explain
README.md:223-320	1,884개 모듈의 분포를 책임 지도로 읽는다	matrix
README.md:17-63	전체 범위 감사는 이렇게 읽는다	walkthrough
README.md:202-220	전체 범위 감사는 이렇게 읽는다	walkthrough

STUDY SYNTHESIS

기준선 검증

읽은 내용을 요약하는 대신, 새로운 변경이나 장애 상황에서도 같은 보장을 찾아낼 수 있는지 확인한다.

검증 질문	feature()가 보이는 것과 활성화의 차이를 설명한다.
검증 질문	build 성공과 semantic equivalence를 구분한다.
검증 질문	OBSERVED와 INFERENCE 문장 하나씩을 직접 작성한다.

이 권의 완료 기준

이후 권에서 어떤 문장도 복원 저장소의 한계를 넘어 공식 운영 사실로 확대하지 않는다.

답에는 반드시 현재 snapshot의 코드 근거, 그 근거에서 도출한 해석, 아직 확인할 수 없는 부분을 구분해 적는다.